



线程池使用建议

北京理工大学计算机学院
金旭亮

设置合适的线程数量



CPU密集型的任务（比如加密），最佳线程数为CPU核数的1~2倍。



IO密集型的任务（比如读写数据库、文件系统、网络等），最佳线程数可以参考以下公式（由Brain Goetz给出）：

线程数 = CPU核心数 * (1 + 平均等待时间/平均工作时间)

线程池的饱和状态

当调用线程向线程池中投递大量任务后，线程池可能就处于饱和状态了。

所谓“饱和状态”，是指当前线程池队列已经满了，并且线程池中的线程已经达到了最大线程个数。



何时线程池会拒绝接收任务？



当线程池关闭时，再向它提交新任务，会被拒绝。



当线程池处于“饱和状态”时，再向它提交新任务，也会被拒绝。

线程池的拒绝策略



当线程池处于饱和状态时，再向线程池投递任务，这时，如何处理投递的任务，由线程池拒绝策略决定，应该依据应用场景，选择合适的拒绝策略。

拒绝策略	策略内容
AbortPolicy	抛弃新增任务，并给调用线程抛出 RejectedExecutionException 异常，这是线程池默认策略
CallerRunsPolicy	使用调用线程执行新提交的任务
DiscardPolicy	默默抛弃新增任务
DiscardOldestPolicy	抛弃线程池队列里面最老的任务，并把新任务添加到线程池进行执行

总结下使用线程池的注意点



避免任务堆积



避免线程数过度增加



在实际开发中，可以通过分析要处理任务的特性，“定制”一个具有特定参数值的线程池，以便高效地处理任务。

自定义线程池示例-1

```
//获取本机CPU核心数
2 usages
private final static int AVALIABLE_PROCESSORS =
    Runtime.getRuntime().availableProcessors();

//自定义线程池
5 usages
private final static ThreadPoolExecutor POOL_EXECUTOR =
    new ThreadPoolExecutor(AVALIABLE_PROCESSORS,
        maximumPoolSize: AVALIABLE_PROCESSORS * 2, keepAliveTime: 1,
        TimeUnit.MINUTES,
        new LinkedBlockingQueue<>( capacity: 5),
        new ThreadPoolExecutor.CallerRunsPolicy());
```

CustomThreadPoolExample.java

自定义线程池示例-2

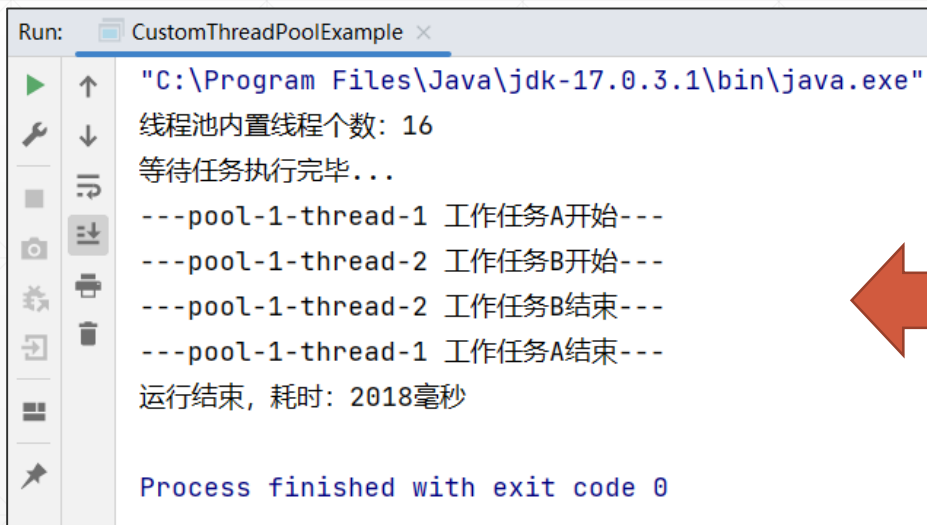
```
public static void doSomethingA() {  
    var threadName : String = Thread.currentThread().getName();  
    System.out.println("---"+threadName+" 工作任务A开始---");  
    try {  
        Thread.sleep( millis: 2000);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    System.out.println("---"+threadName+" 工作任务A结束---");  
}
```

```
public static void doSomethingB() {  
    var threadName : String = Thread.currentThread().getName();  
    System.out.println("---"+threadName+" 工作任务B开始---");  
    try {  
        Thread.sleep( millis: 2000);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    System.out.println("---"+threadName+" 工作任务B结束---");  
}
```


自定义线程池示例-3

针对特定的应用场景，选择自定义线程池是提升程序性能的有效手段。

```
public static void main(String[] args) throws InterruptedException {
    System.out.println("线程池内置线程个数: " + POOL_EXECUTOR.getCorePoolSize());
    long start = System.currentTimeMillis();
    // 异步执行任务A和B
    POOL_EXECUTOR.execute(() -> {
        try {
            doSomethingA();
        } catch (Exception e) {
            e.printStackTrace();
        }
    });
    POOL_EXECUTOR.execute(() -> {
        try {
            doSomethingB();
        } catch (Exception e) {
            e.printStackTrace();
        }
    });
    //关闭线程池，这时，会等待线程完成当前的任务
    POOL_EXECUTOR.shutdown();
    System.out.println("等待任务执行完毕...");
    // 3.同步等待线程A运行结束
    POOL_EXECUTOR.awaitTermination( timeout: 10, TimeUnit.SECONDS);
    var workingTime = System.currentTimeMillis() - start;
    System.out.println("运行结束，耗时: " + workingTime + "毫秒");
}
```



```
Run: CustomThreadPoolExample x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
线程池内置线程个数: 16
等待任务执行完毕...
---pool-1-thread-1 工作任务A开始---
---pool-1-thread-2 工作任务B开始---
---pool-1-thread-2 工作任务B结束---
---pool-1-thread-1 工作任务A结束---
运行结束，耗时: 2018毫秒

Process finished with exit code 0
```